

Validation and Measurements Report

Hand-in 3

March 5, 2026

1 Validation and Measurements

1.1 Verify signature generation and verification

We can generate and verify a signature on a message.

- **Positive test:** Sign a message, verify with the same message -> passes.
- **Negative test:** Modify the message and verify again -> verification fails.

See `rsa/signature_test.go` - `TestVerifySignature`.

Run: `cd rsa && go test -run TestVerifySignature -v`

1.2 Hashing speed (bits per second)

- **Message size:** 10 KB = 10,000 bytes = 80,000 bits
- **Measured:** 4,200 ns per 10 KB hash
- **Throughput:** $80,000 / (4,200 \times 10^{-9}) = 1.95 \times 10^{10}$ bits/s = **19 Gbit/s**

Run: `cd rsa && go test -bench=BenchmarkHashSpeed -benchmem`

1.3 RSA signature generation time (2000-bit key)

Signing a hash value (256 bits) with a 2000-bit RSA key:

- **Time per signature:** 1,970,926 ns = **1.97 ms**

Run: `cd rsa && go test -bench=BenchmarkSignatureOnHash -benchmem`

1.4 RSA throughput vs hashing

If RSA were used directly on the whole message: at most 2000 bits per RSA operation.
With ~1.97 ms per op:

- **RSA-direct throughput:** $2000 / 0.00197 = 1.14 \times 10^6$ bits/s = **1.14 Mbit/s**

Hashing (from item 2): ~19 Gbit/s.

Hashing is far faster. With **hash-then-sign** we sign the 256-bit hash once per message; the cost of hashing is negligible compared to RSA. **Hashing greatly improves efficiency.**

2 How to run tests and the project

Run all RSA tests (key size, encrypt/decrypt, signatures, AES, hybrid):

```
cd rsa
go test
```

Run signature verification only:

```
cd rsa
go test -run TestVerifySignature -v
```

Run hashing speed benchmark (10 KB, ~4,200 ns -> ~19 Gbit/s):

```
cd rsa
go test -bench=BenchmarkHashSpeed -benchmem
```

Run RSA signature benchmark (2000-bit key, ~1.97 ms per signature):

```
cd rsa
go test -bench=BenchmarkSignatureOnHash -benchmem
```

Run peer (P2P flooding + ledger) tests:

```
cd peer
go test
```

Run the peer demo (5 peers, 5 transactions each, ledger convergence check):

```
cd peer
go run .
```